Instituto Politécnico Nacional

Escuela Superior de Cómputo

Ingeniería en Inteligencia Artificial

Algoritmos Bioinspirados

Sistema de Planificación de Tareas mediante Algoritmos Genéticos

Autores: Gonzalez Pérez Monserrat y Zamudio López Leonardo.

Grupo: 5BM2

Índice

1. Marco teórico de algoritmos bioinspirados **3**

1.1. Introducción . . . 3

1.2. Algoritmos genéticos . . . 3

1.2.1. Representación cromosómica . . . 4

1.2.2. Función de adaptación . . . 4

1.2.3. Selección . . . 4

1.2.4. Cruce . . . 5

1.2.5. Mutación . . . 5

1.2.6. Convergencia y diversidad . . . 5

1.3. Algoritmos bioinspirados relacionados . . . 5

2. Fundamentos matemáticos de la evolución artificial **6**

2.1. Población, genotipo y fitness . . . 6

2.2. Selección como presión evolutiva . . . 7

2.3. Cruce como mezcla de material genético . . . 7

2.4. Mutación como fuente de diversidad . . . 7

2.5. Diversidad, entropía y convergencia prematura . . . 8

2.6. Teoría intuitiva de la exploración y la explotación . . . 8

3. Definición del problema **9**

3.1. Problema de programación de tareas . . . 9

3.2. Modelado matemático . . . 9

3.3. Naturaleza del problema abordado por el proyecto . . . 10

4. Aplicación del algoritmo genético al problema de programación de tareas **10**
4.1. Codificación de una solución . . . 10
4.2. Decodificación de la solución . . . 11
4.3. Fitness y evaluación . . . 11
4.4. Operadores evolutivos en el problema . . . 11

5. Arquitectura interna del sistema **12**
5.1. Visión general . . . 12
5.2. Frontend . . . 12
5.3. Backend REST y controladores . . . 13
5.4. Modelo del problema y persistencia . . . 13
5.5. Celery y ejecución asíncrona . . . 14
5.6. Representación del escenario JSON . . . 14

6. Formalización del makespan y la decodificación **15**
6.1. Definición de variables . . . 15
6.2. Regla de decodificación . . . 15
6.3. Makespan . . . 15

7. Pseudocódigo riguroso del algoritmo genético aplicado **16**

8. Conclusiones **18**

Anexos: Ejemplos de prueba y diagramas de Gantt **19**
Definición de los ejemplos incluidos . . . 19
Escenario 1 . . . 20
Escenario 2 . . . 21
Escenario 3 . . . 22

Resumen

Este trabajo presenta el marco teórico y técnico de un sistema para la resolución del problema de planificación de tareas mediante algoritmos genéticos. El problema se modela como una instancia del Job Shop Scheduling Problem, donde cada trabajo está compuesto por operaciones con precedencias internas y recursos compartidos. El objetivo del sistema es minimizar el tiempo total de ejecución, medido mediante el makespan, bajo restricciones de precedencia y disponibilidad de máquinas. Para ello, se propone un algoritmo evolutivo que codifica soluciones como permutaciones de operaciones, evalúa su calidad mediante una función objetivo y aplica operadores de selección, cruce y mutación para mejorar iterativamente la población. La arquitectura del sistema está organizada en frontend, backend REST y capa de cómputo asíncrono con Celery, apoyada en MongoDB y Redis. El documento presenta la formalización matemática del problema, la lógica de decodificación de una solución, la estructura del algoritmo genético y el flujo de ejecución del sistema desarrollado.

Palabras clave: optimización combinatoria, planificación de tareas, Job Shop Scheduling, algoritmos genéticos, bioinspiración, makespan, Celery, Flask, React.

1 Marco teórico de algoritmos bioinspirados

1.1 Introducción

Los algoritmos bioinspirados son métodos computacionales inspirados en procesos naturales, tales como la evolución biológica, el comportamiento de enjambres, la adaptación de sistemas inmunológicos o la dinámica de colonias de insectos. Su objetivo no es reproducir exactamente la biología, sino abstraer principios generales que permitan resolver problemas complejos cuando los métodos clásicos resultan poco prácticos.

Los algoritmos bioinspirados se distinguen por:

- explorar espacios de soluciones de alta dimensionalidad;
- usar poblaciones de candidatos, no una única solución;
- combinar exploración y explotación;
- tolerar ruido y no convexidad;
- adaptarse mediante iteraciones de mejora.

Dentro de esta categoría, los algoritmos genéticos son uno de los paradigmas más representativos, tal como se expone en la formulación clásica de Goldberg y en los tratados de Eiben y Smith sobre computación evolutiva [**? ?**]. Asimismo, la referencia de curso de Coello Coello aporta una visión didáctica de la computación evolutiva y del marco conceptual de la optimización bioinspirada [1].

1.2 Algoritmos genéticos

Un algoritmo genético (AG) es una técnica de optimización evolutiva que imita la evolución natural por selección, reproducción y variación genética. Se parte de una población inicial de soluciones candidatas, cada una codificada como un cromosoma.

Cada individuo representa una posible solución al problema. El algoritmo asigna a cada individuo una aptitud o fitness, que mide qué tan buena es la solución conforme a la función objetivo. A partir de allí, se aplican operadores evolutivos:

1. **Selección**: elegir individuos con mayor fitness para reproducirse.
2. **Cruce**: combinar dos cromosomas para generar descendencia.
3. **Mutación**: alterar aleatoriamente parte del cromosoma para introducir diversidad.
4. **Reemplazo**: sustituir parte de la población por la nueva generación.

La evolución se repite durante varias generaciones hasta alcanzar un criterio de parada, por ejemplo, un número máximo de iteraciones, convergencia del fitness o tiempo máximo.

1.2.1. Representación cromosómica

Una solución se representa por un vector o cadena de genes. En un problema de programación, cada gen puede codificar una operación, una máquina o un orden relativo entre tareas. Un cromosoma puede representar una permutación de operaciones o una asignación de prioridad.

Si un cromosoma es $x = (x_1, x_2, \ldots, x_L)$, con L genes, la posición del gen determina el orden de ejecución o la prioridad de una operación en la secuencia.

1.2.2. Función de adaptación

Dada una solución x, el algoritmo evalúa su calidad mediante una función de fitness $f(x)$ que normalmente se construye a partir de la función objetivo del dominio. En el caso de minimización, suele usarse

$$F(x) = \frac{1}{1 + f(x)}$$

para convertir una función objetivo no acotada en un valor proporcional a la calidad. En el problema de programación de tareas, la función objetivo puede ser el makespan:

$$F(x) = \frac{1}{1 + C_{\text{máx}}(x)}.$$

Esto hace que los individuos con menor makespan tengan mayor fitness.

1.2.3. Selección

La selección favorece la reproducción de soluciones mejores. Un método clásico es la selección por torneo. Dados dos o más individuos, se comparan sus fitness y se selecciona el mejor. También existen la ruleta proporcional o la selección por ranking.

La idea es mantener una presión selectiva sin destruir por completo la diversidad. Formalmente, si la población es P_t, la selección define una probabilidad de elección

$$P(i \in P_t) = \frac{F(i)}{\sum_{j \in P_t} F(j)}.$$

1.2.4. Cruce

El cruce combina información genética de dos progenitores para producir descendencia. En problemas de permutación, se usan operadores como:

- Order Crossover (OX),
- Partially Mapped Crossover (PMX),
- Position-Based Crossover,
- Cycle Crossover (CX).

El objetivo es preservar parcialmente el orden de las operaciones y generar soluciones factibles. Supongamos que dos padres son $p_1 = (a, b, c, d, e)$ y $p_2 = (e, c, a, d, b)$. Un cruce por orden toma una sección de referencia de un progenitor y rellena el resto respetando el orden de aparición del otro progenitor.

1.2.5. Mutación

La mutación introduce variabilidad aleatoria para evitar la convergencia prematura. En problemas de permutación, puede consistir en:

- intercambio de dos posiciones;
- inversión de un bloque;
- desplazamiento de un elemento a otra posición.

Si x es el cromosoma y i, j dos índices, una operación de intercambio es:

$$x_i \leftrightarrow x_j.$$

Con esto se altera parcialmente la estructura del individuo sin perder la representación válida.

1.2.6. Convergencia y diversidad

La evolución no solo debe mejorar la calidad promedio, sino también mantener diversidad. Si la población se acerca demasiado rápidamente a una misma solución, el algoritmo puede quedar atrapado en un óptimo local. Por ello, las tasas de mutación, la selección y la estrategia de reemplazo son parámetros críticos.

1.3 Algoritmos bioinspirados relacionados

Además de los algoritmos genéticos, existen otros enfoques bioinspirados ampliamente usados en optimización:

- **Optimización por colonia de hormigas (ACO)**: inspira el comportamiento de búsqueda de caminos por parte de las hormigas. Las hormigas depositan feromonas; los caminos con menores costes reciben mayor concentración de feromonas.

- **Optimización por enjambres de partículas (PSO)**: cada partícula mantiene posición y velocidad; la mejor posición personal y la mejor global influyen en la evolución.
- **Algoritmos de enjambre** y **simulated annealing**: usan analogías con física y comportamiento colectivo.

Todos estos enfoques comparten el principio de buscar soluciones aceptables en espacios complejos, sin depender de derivadas o de la estructura convexa del problema.

2 Fundamentos matemáticos de la evolución artificial

2.1 Población, genotipo y fitness

Un algoritmo genético trabaja con una población de soluciones candidatas, no con una única solución. Sea

$$P_t = \{x_1^{(t)}, x_2^{(t)}, \ldots, x_N^{(t)}\},$$

la población en la generación t, donde cada individuo $x_i^{(t)}$ representa una posible solución del problema. En el contexto de la programación de tareas, cada individuo es un cromosoma que codifica una asignación o una secuencia de prioridad para las operaciones. El valor asociado a cada cromosoma se mide mediante una función de evaluación $f(x_i^{(t)})$, que normalmente corresponde a una medida de costo o penalización.

En la mayoría de los problemas de optimización, el objetivo es minimizar un costo. Por ejemplo, en el problema de scheduling, el costo puede ser el makespan:

$$f(x) = C_{\text{máx}}(x).$$

Sin embargo, para incorporar la selección natural de forma más conveniente, se define un valor de adaptación o fitness, que transforma la función objetivo en un criterio de preferencia. Una elección clásica es:

$$F(x) = \frac{1}{1 + f(x)},$$

cuando se desea minimizar $f(x)$. Esta transformación cumple dos propósitos: hace que los individuos con mejor calidad tengan mayor fitness, y evita dividir entre cero cuando el costo es nulo o muy pequeño.

La evolución artificial puede describirse como una secuencia recursiva de poblaciones:

$$P_0 \xrightarrow{\text{selección}} P_1 \xrightarrow{\text{cruce y mutación}} P_2 \to \cdots \to P_T,$$

donde cada generación se obtiene a partir de la anterior mediante operadores de selección, reproducción y variación. La idea central es que la población no es estática: cada iteración conserva parte de la información útil y agrega diversidad para explorar nuevas soluciones.

2.2 Selección como presión evolutiva

La selección es el mecanismo que favorece a los individuos de mejor calidad. Dada una población P_t, el algoritmo asigna una probabilidad de supervivencia o reproducción a cada individuo en función de su fitness. En una versión proporcional, la probabilidad de elegir un cromosoma x_i es:

$$\Pr(x_i \text{ es elegido}) = \frac{F(x_i)}{\sum_{j=1}^{N} F(x_j)}.$$

La selección introduce una presión evolutiva, porque los individuos con mejor rendimiento tienen más oportunidades de reproducirse, mientras que los peores tienden a desaparecer. Sin embargo, si la presión es excesiva, el algoritmo puede converger prematuramente hacia un buen pero no óptimo local. Por ello, la selección no debe ser ni demasiado débil ni demasiado fuerte.

En la práctica, existen diversos métodos de selección, como la selección por torneo, la ruleta proporcional y la selección por ranking. En este proyecto se usa selección por torneo, donde un conjunto pequeño de individuos compite y se selecciona el mejor de ese subconjunto. Esta estrategia es especialmente útil en problemas combinatorios, porque reduce el costo computacional y mantiene un equilibrio razonable entre explotación y exploración.

2.3 Cruce como mezcla de material genético

El operador de cruce combina la información de dos progenitores para producir descendencia. Formalmente, si $x^{(p_1)}$ y $x^{(p_2)}$ son dos cromosomas, el cruce define una operación

$$\mathcal{C}(x^{(p_1)}, x^{(p_2)}) = (x^{(h_1)}, x^{(h_2)}),$$

que produce dos hijos con características heredadas de ambos padres. En problemas de permutación o secuenciación, no basta con combinar bits arbitrarios, porque se deben preservar propiedades de factibilidad, como la ausencia de duplicados o la coherencia de precedencias.

Por esta razón, en programación de tareas suelen usarse operadores de cruce especializados, como order crossover (OX), partially mapped crossover (PMX) o variantes de intercambio segmentado. La idea general es conservar fragmentos útiles de un progenitor y completar la información faltante con la estructura del otro padre, manteniendo la viabilidad de la solución.

En términos intuitivos, el cruce hace que la población «aprenda» combinando componentes exitosos de soluciones previas. Si las dos soluciones parentales tienen buena calidad, la recombinación puede producir hijos con mejor rendimiento al combinar rasgos válidos de ambos.

2.4 Mutación como fuente de diversidad

La mutación es el operador responsable de introducir variación aleatoria en la población. Su finalidad es evitar que el algoritmo quede atrapado en un óptimo local, especialmente cuando la mayoría de los individuos empiezan a ser muy parecidos entre sí. Si una población pierde diversidad, la selección se vuelve dominante y la exploración desaparece.

En un cromosoma $x = (x_1, x_2, \ldots, x_L)$, una mutación puede consistir en intercambiar dos genes, invertir un bloque de genes o alterar una posición con cierta probabilidad. Por ejemplo, una operación de intercambio entre dos posiciones i y j se representa como:

$$x_i \leftrightarrow x_j.$$

La mutación no debe aplicarse con demasiada frecuencia, porque entonces el algoritmo se comporta como una búsqueda aleatoria. Pero tampoco debe ser demasiado rara, porque en ese caso la diversidad desaparece y la población converge prematuramente. Por eso se usa una tasa de mutación p_m que controla la intensidad del cambio aleatorio.

2.5 Diversidad, entropía y convergencia prematura

La diversidad de una población es un concepto clave para evaluar si el algoritmo sigue explorando soluciones distintas. Una medida simple, aunque no única, es la distancia promedio entre cromosomas:

$$D(P_t) = \frac{2}{N(N-1)} \sum_{i<j} d(x_i^{(t)}, x_j^{(t)}),$$

donde $d(\cdot, \cdot)$ representa una distancia apropiada entre individuos. En problemas de permutación, esta distancia puede interpretarse como la discrepancia entre órdenes de prioridad o la cantidad de diferencias entre posiciones de operaciones.

Cuando $D(P_t)$ baja significativamente, la población tiende a encogerse alrededor de soluciones similares. Este fenómeno se conoce como convergencia prematura. En términos de optimización, puede ser problemático porque el algoritmo deja de explorar y se queda en una región local del espacio de soluciones. Por ello, la combinación entre selección, cruces y mutación debe mantener una diversidad mínima suficiente durante varias generaciones.

La convergencia, por otra parte, es deseable en la medida en que la mejor solución encontrada mejora progresivamente. Se dice que el algoritmo converge cuando la calidad de la población mejora y la mejor solución observada se estabiliza. Formalmente, si $M_t = \min_{x \in P_t} f(x)$, la convergencia se observa cuando M_t deja de disminuir de forma significativa a lo largo de varias generaciones. Sin embargo, la convergencia no garantiza optimalidad global; en la práctica, la meta es encontrar soluciones suficientemente buenas dentro de un presupuesto computacional razonable.

2.6 Teoría intuitiva de la exploración y la explotación

La evolución artificial se apoya en un equilibrio entre dos dinámicas complementarias:

- **Exploración**: la capacidad de visitar regiones distintas del espacio de soluciones, principalmente a través de la mutación y de la recombinación de individuos distintos.
- **Explotación**: la capacidad de reforzar soluciones ya prometedoras mediante la selección y la reproducción de individuos con mejor fitness.

Si el algoritmo explora demasiado, puede perder rápidamente soluciones prometedoras y requerir más tiempo para encontrar buenas soluciones. Si explota demasiado, corre el riesgo de caer en óptimos locales. La estrategia efectiva es adaptar la presión selectiva y la tasa de mutación para mantener una población equilibrada.

En problemas como el Job Shop Scheduling Problem, esta idea es especialmente relevante, porque el espacio de soluciones es enorme, discreto y altamente no convexo. No existe una trayectoria lineal hacia la solución óptima, y la forma del paisaje de fitness cambia conforme se modifican las asignaciones y los tiempos de procesamiento. Por eso, los algoritmos genéticos resultan adecuados: ofrecen una estrategia robusta para explorar combinaciones complejas sin depender de gradientes ni de convexidad.

3 Definición del problema

3.1 Problema de programación de tareas

El problema de planificación de tareas puede interpretarse como una instancia del Job Shop Scheduling Problem (JSP), aunque también existe la variante de Flow Shop en algunos contextos. Dado un conjunto de trabajos $J = \{J_1, \ldots, J_n\}$, cada trabajo J_j está compuesto por una secuencia de operaciones

$$O_{j1}, O_{j2}, \ldots, O_{j,m_j},$$

donde cada operación O_{jk} requiere una máquina específica $M_{jk} \in \mathcal{M}$ durante un tiempo de procesamiento $p_{jk,m}$.

Cada trabajo puede requerir varias máquinas y el orden de las operaciones dentro del trabajo es fijo. En consecuencia, el problema consiste en decidir la secuencia cronológica de ejecución de las operaciones para satisfacer las siguientes condiciones:

- cada operación se ejecuta en una única máquina;
- cada máquina procesa una sola operación a la vez;
- el orden dentro de cada trabajo es respetado;
- el objetivo es minimizar alguna métrica global, por ejemplo, el makespan.

La métrica más frecuente es el makespan, definido como:

$$C_{\text{máx}} = \max_{j,k} C_{jk},$$

donde C_{jk} es el instante de finalización de la operación O_{jk}. El objetivo es resolver

$$\min C_{\text{máx}}$$

sujeto a la lógica de precedencia y exclusión mutua de recursos.

3.2 Modelado matemático

Una formulación clásica del JSP se basa en variables binarias x_{ijk} que indican si la operación O_{jk} es ejecutada antes que la operación $O_{i\ell}$ en una máquina dada, o en su representación más convencional con variables de inicio S_{jk} y restricciones de precedencia.

Para cada operación O_{jk}:

$$S_{jk} \geq 0,$$

y para cada par de operaciones que comparten misma máquina:

$$S_{jk} + p_{jk} \leq S_{i\ell} + M(1 - x_{jk,i\ell}),$$

$$S_{i\ell} + p_{i\ell} \leq S_{jk} + M x_{jk,i\ell},$$

donde M es una cota suficientemente grande y $x_{jk,i\ell} \in \{0, 1\}$ decide el orden relativo. Además, para las precedencias internas del trabajo:

$$S_{j,k+1} \geq S_{jk} + p_{jk},$$

con la función objetivo:

$$\min \max_{j,k} (S_{jk} + p_{jk}).$$

Este problema pertenece a la clase de problemas combinatorios NP-hard y, en general, no admite solución exacta eficiente para instancias medianas y grandes. Por esa razón, se recurre a métodos metaheurísticos como los algoritmos genéticos.

3.3 Naturaleza del problema abordado por el proyecto

En el sistema implementado, cada escenario de prueba se representa por:

- un conjunto de trabajos;
- para cada trabajo, una secuencia de operaciones;
- tiempos de procesamiento por máquina;
- una colección de máquinas disponibles.

La representación del JSON del proyecto corresponde a un formato conveniente para cargar escenarios, donde un trabajo contiene una lista de operaciones y cada operación define un diccionario de tiempos asociados a cada máquina. Este formato se transforma en una representación interna de programación para poder evaluar la calidad de distintas secuencias.

El objetivo es encontrar una configuración de orden de operaciones que minimice el tiempo total de finalización. Este tipo de problema exige manejar la complejidad combinatoria, la no convexidad y la existencia de múltiples soluciones factibles con distinto grado de calidad.

4 Aplicación del algoritmo genético al problema de programación de tareas

4.1 Codificación de una solución

En este proyecto, cada instancia se modela con un conjunto de trabajos y operaciones. Una solución candidata debe codificar una secuencia global de ejecución de operaciones. La codificación puede ser de tipo permutación, por ejemplo:

$$\pi = (3, 1, 5, 2, 4, \dots)$$

donde cada número identifica una operación y el orden de aparición representa la prioridad de ejecución.

Con una codificación de este tipo, se reconstruye la programación mediante una lógica de simulación de flujo de trabajo: se recorre la secuencia, se asigna cada operación a la máquina requerida y se calcula el inicio y fin de la operación según la disponibilidad de la máquina y la precedencia del trabajo.

4.2 Decodificación de la solución

La decodificación transforma una permutación en una programación factible. Para cada operación O_{jk}:

- se consulta la máquina requerida;
- se obtiene el tiempo de procesamiento p_{jk};
- se calcula el instante de inicio S_{jk} como el máximo entre:
  - la finalización de la operación anterior del mismo trabajo;
  - la disponibilidad de la máquina.
- se obtiene el final $C_{jk} = S_{jk} + p_{jk}$.

Esto se puede formalizar como:

$S_{jk} = \max\{\text{Fin de la operación precedente en el trabajo, Fin de la última operación en la máquina}\}$.

Al final, el makespan se calcula como:

$$C_{\max} = \max_{j,k} C_{jk}.$$

4.3 Fitness y evaluación

Dado un cromosoma x, la evaluación consiste en decodificarlo, construir el cronograma y calcular $C_{\max}(x)$. Si se usa el criterio de minimización, el fitness queda definido como:

$$F(x) = \frac{1}{1 + C_{\max}(x)}.$$

Las soluciones con menor makespan reciben mayor fitness y tienen mayor probabilidad de ser seleccionadas.

4.4 Operadores evolutivos en el problema

En el proyecto, el algoritmo genético aplicado incluye:

- población inicial aleatoria;
- cálculo del makespan por cada individuo;
- selección de individuos con mejor rendimiento;

- cruce entre padres para formar descendencia;
- mutación mediante intercambio o reordenamiento de genes;
- evaluación iterativa de la nueva generación.

El resultado final es un conjunto de soluciones mejoradas, así como una serie de valores de convergencia, que muestran la evolución del makespan a lo largo de las generaciones.

5 Arquitectura interna del sistema

5.1 Visión general

El sistema desarrollado se organiza como una arquitectura distribuida en capas, donde cada componente tiene una responsabilidad bien definida. Esta división no es solo una decisión de diseño académico, sino una necesidad funcional del proyecto: la resolución del problema de programación de tareas exige carga computacional intensa y un flujo de trabajo que no bloquee la interfaz del usuario. Por ello, la aplicación se separa en una capa de interacción, una capa de lógica de negocio, una capa de ejecución asíncrona y una capa de persistencia.

En términos prácticos, el proyecto cuenta con cuatro bloques principales:

1. **Capa de presentación**: la aplicación React + Vite, responsable de cargar escenarios, iniciar ejecuciones, consultar resultados y mostrar gráficas y tablas.
2. **Capa de servicio y API**: el backend en Flask, que valida entradas, coordina ejecuciones y expone los endpoints REST.
3. **Capa de cómputo**: los workers de Celery, encargados de ejecutar la lógica del algoritmo genético y construir el cronograma final.
4. **Capa de infraestructura**: MongoDB, Redis y Docker Compose, que gestionan la persistencia, el encolado de tareas y la ejecución del entorno.

La separación funcional es crítica, porque el frontend no ejecuta el algoritmo, sino que consume resultados producidos por el backend. Esto permite mantener una interfaz reactiva y una experiencia de usuario fluida, sin congestionar la navegación ni bloquear la aplicación durante tareas que pueden tardar varios segundos o minutos.

5.2 Frontend

El frontend está implementado con React y TypeScript y se comunica con el backend mediante peticiones HTTP a una API REST. Su función principal es actuar como interfaz entre el usuario y el sistema de optimización, pero sin contener la lógica compleja del algoritmo. Su responsabilidad se concentra en tres tareas fundamentales:

- cargar escenarios desde archivos JSON o desde la base de datos;
- configurar parámetros de ejecución, como tamaño de población, número de generaciones o tasa de mutación;
- visualizar el estado de la ejecución y los resultados finales mediante tablas, gráficos de convergencia y diagramas de Gantt.

Entre sus vistas principales se encuentran:

- carga de escenarios;
- listado de escenarios almacenados;
- panel de ejecución con parámetros del algoritmo;
- panel de reportes estadísticos;
- gráficos de convergencia y diagramas de Gantt.

Desde el punto de vista de arquitectura, el frontend funciona como cliente ligero. Esto es ventajoso porque mantiene la lógica pesada en el servidor y evita duplicar el cálculo en la parte visual. La representación de un escenario se envía al backend en formato JSON, y la respuesta se presenta al usuario con una estructura de resultados ya procesada.

5.3 Backend REST y controladores

El backend se implementa con Flask y expone una API bajo el prefijo ‘/api/v1‘. Su objetivo es centralizar la lógica de negocio y actuar como punto de entrada para todas las operaciones del sistema. Los controladores principales son:

- `scenario_controller`: se encarga de registrar, listar y consultar escenarios;
- `execution_controller`: inicia ejecuciones, consulta su estado y recupera resultados;
- `report_controller`: genera reportes agregados y estadísticas del rendimiento del algoritmo.

Cada solicitud HTTP llega al controlador correspondiente, se valida la forma del payload, y luego se delega el procesamiento a servicios especializados. Por ejemplo, al recibir un escenario nuevo, el backend lo valida, lo transforma a una estructura interna y lo almacena en MongoDB. Cuando el usuario lanza una ejecución, el backend crea una tarea asíncrona en Celery y devuelve un identificador de trabajo para que el cliente pueda consultar su estado más adelante.

Esta separación entre API y servicios es importante porque permite mantener una interfaz clara, reutilizar lógica y evitar mezclar la capa de transporte con la lógica del algoritmo. El controlador responde a la solicitud externa, pero la ejecución real del cálculo queda fuera de la ruta HTTP.

5.4 Modelo del problema y persistencia

El modelo de dominio del proyecto se representa en MongoDB mediante MongoEngine. El almacenamiento se realiza en dos niveles: por un lado, los escenarios, y por otro, los resultados de las ejecuciones. Un escenario contiene la información estructural del problema, como trabajos, operaciones, tiempos de procesamiento por máquina y restricciones de precedencia. Una ejecución, por su parte, registra:

- identificador del escenario;
- parámetros del algoritmo;
- estado de la ejecución (‘PENDING‘, ‘RUNNING‘, ‘COMPLETED‘, ‘FAILED‘);

- resultados temporales y finales;
- datos para visualización y estadísticas.

La persistencia cumple varias funciones dentro del sistema. En primer lugar, permite reutilizar escenarios ya definidos sin tener que volver a cargarlos desde archivos. En segundo lugar, conserva la trazabilidad de cada ejecución para compararlas entre sí. En tercer lugar, facilita la generación de reportes con métricas del comportamiento del algoritmo, como evolución del makespan, mejor solución encontrada y tiempos de ejecución. Finalmente, sirve como mecanismo de recuperación ante fallos o reinicios del sistema.

5.5 Celery y ejecución asíncrona

El proyecto usa Celery como motor de procesamiento asíncrono junto con Redis como broker y backend de resultados. Esto es esencial porque la optimización mediante algoritmo genético puede requerir varias iteraciones y varios segundos, o incluso minutos, dependiendo del tamaño del problema. Si dichas operaciones se ejecutaran directamente en la petición HTTP, la interfaz del usuario se congelaría y la experiencia de uso empeoraría sustancialmente.

El flujo de ejecución real es el siguiente:

1. el usuario carga o selecciona un escenario desde el frontend;
2. el backend valida el contenido y lo persiste en MongoDB;
3. el cliente solicita una ejecución con los parámetros del algoritmo;
4. el backend encola una tarea en Redis mediante Celery;
5. el worker de Celery toma la tarea, ejecuta la lógica del algoritmo genético y calcula el makespan;
6. el estado y los resultados se actualizan en la base de datos;
7. el cliente consulta periódicamente el estado o recupera los resultados finales.

Esta arquitectura presenta varias ventajas. En primer lugar, desacopla la interfaz del cálculo. En segundo lugar, permite gestionar varias ejecuciones simultáneamente. En tercer lugar, mejora la tolerancia a fallos y evita bloquear al usuario durante una optimización de larga duración. En la práctica, esto convierte al sistema en una aplicación de tipo pipeline: entrada del problema, encolado de la tarea, resolución computacional y entrega de resultados.

5.6 Representación del escenario JSON

La estructura del escenario se representa como un conjunto de trabajos, cada uno con una lista de operaciones, y cada operación con los tiempos de procesamiento asociados a diferentes máquinas. Este formato es legible para el usuario y también sirve como entrada a la lógica de optimización. Una operación típica tiene la forma:

```
{
  "op_id": 1,
  "times": {
    "M1": 5.5,
```

```
    "M2": 4.2,
    "M3": 7.6
  }
}
```

La lógica del algoritmo interpreta esta estructura como un conjunto de restricciones de precedencia y disponibilidad de recursos. Aunque el usuario visualiza un JSON de fácil lectura, internamente el sistema transforma esa estructura en una representación más compacta para la ejecución del algoritmo. De esta forma, la capa de presentación se mantiene independiente de la complejidad computacional del problema y el núcleo evolutivo opera sobre representaciones optimizadas para cálculo y evaluación.

6 Formalización del makespan y la decodificación

6.1 Definición de variables

Sea $J = \{J_1, \ldots, J_n\}$ el conjunto de trabajos y $\mathcal{M} = \{M_1, \ldots, M_m\}$ el conjunto de máquinas. Cada trabajo J_j tiene operaciones $O_{j1}, \ldots, O_{j,m_j}$, ordenadas por precedencia interna. La operación O_{jk} requiere la máquina $\mu(j,k)$ y tiene duración p_{jk}.

Se define:

- S_{jk}: instante de inicio de la operación O_{jk};
- C_{jk}: instante de finalización de la operación O_{jk};
- $pred(j,k)$: operación previa del mismo trabajo;
- $last(M_r)$: última operación asignada a la máquina M_r según la secuencia construida.

6.2 Regla de decodificación

Dada una permutación π de operaciones, la lógica de programación se recorre en ese orden. Para cada operación O_{jk} en π:

$$S_{jk} = \max\left(\mathrm{Fin}(pred(j,k)),\ \mathrm{Fin}(last(\mu(j,k)))\right),$$

$$C_{jk} = S_{jk} + p_{jk}.$$

Aquí, $\mathrm{Fin}(pred(j,k))$ representa la finalización de la operación anterior del mismo trabajo, y $\mathrm{Fin}(last(\mu(j,k)))$ representa la disponibilidad de la máquina requerida.

Si la operación es la primera de su trabajo, entonces se toma $\mathrm{Fin}(pred(j,k)) = 0$. Si la máquina todavía está libre, entonces $\mathrm{Fin}(last(\mu(j,k))) = 0$.

6.3 Makespan

La métrica global asociada a la solución es:

$$C_{\max}(\pi) = \max_{j,k} C_{jk}.$$

El problema puede formularse como:

$$\min_{\pi} C_{\max}(\pi).$$

Esto convierte la secuencia de operaciones en una variable de decisión y la evaluación del cromosoma en un coste computable y comparable. Dado que el objetivo es minimizar un valor de finalización, la función de fitness suele definirse en forma inversa para facilitar la selección:

$$F(\pi) = \frac{1}{1 + C_{\max}(\pi)}.$$

La solución con menor makespan arroja mayor fitness, lo que favorece su supervivencia en la población.

7 Pseudocódigo riguroso del algoritmo genético aplicado

Sea $\mathcal{O} = \{0, \ldots, O-1\}$ el conjunto de operaciones del escenario, $\mathcal{M} = \{0, \ldots, M-1\}$ la colección de máquinas y $\mathbf{P} \in \mathbb{Z}^{N \times O}$ la población actual, donde cada fila $\mathbf{x}^{(c)}$ representa un cromosoma. Cada componente $x_o^{(c)}$ indica la máquina asignada a la operación o en ese cromosoma. La función de evaluación usa la decodificación por orden de niveles de operaciones y calcula el makespan.

Algorithm 1 Algoritmo genético implementado para la planificación de tareas

Requiere escenario S, tamaño de población N, generaciones G, tasa de mutación p_m, semilla s
Garantiza mejor individuo encontrado $\mathbf{x}^\star$ y su makespan $C^\star_{\max}$

Establecer semilla aleatoria `InicializarSemilla`(s)
Inicializar $\mathbf{P}_0 \in \mathbb{Z}^{N \times O}$ con valores aleatorios en $\{0, \ldots, M-1\}$
$\mathbf{F}_0 \leftarrow$ `EvaluarMakespan`$(\mathbf{P}_0, S)$
$i^\star \leftarrow \arg\min_i \mathbf{F}_0[i]$
$\mathbf{x}^\star \leftarrow \mathbf{P}_0[i^\star]$
$C^\star_{\max} \leftarrow \mathbf{F}_0[i^\star]$
$H \leftarrow [C^\star_{\max}]$
para $g = 1, \ldots, G$ **hacer**
    $\mathbf{P}_{sel} \leftarrow$ `SeleccionarPorTorneo`$(\mathbf{P}_{g-1}, \mathbf{F}_{g-1}, k = 3)$
    $\mathbf{P}_{cruce} \leftarrow$ `CruceUniforme`$(\mathbf{P}_{sel})$
    $\mathbf{P}_g \leftarrow$ `MutacionBalanceadaPorCarga`$(\mathbf{P}_{cruce}, S, p_m)$
    $\mathbf{F}_g \leftarrow$ `EvaluarMakespan`$(\mathbf{P}_g, S)$
    $i_g \leftarrow \arg\min_i \mathbf{F}_g[i]$
    si $\mathbf{F}_g[i_g] < C^\star_{\max}$ **entonces**
        $\mathbf{x}^\star \leftarrow \mathbf{P}_g[i_g]$
        $C^\star_{\max} \leftarrow \mathbf{F}_g[i_g]$
    fin si
    $H.append(C^\star_{\max})$
fin para
devolver $\mathbf{x}^\star, C^\star_{\max}, H$

La rutina de evaluación se define de la siguiente manera:

Algorithm 2 EvaluarMakespan

Requiere población $\mathbf{P}$, escenario S
Garantiza vector de makespans $\mathbf{F}$

para $c = 1, \ldots, N$ **hacer**
  tiempoFinTrabajo $\leftarrow \mathbf{0}_{|J|}$
  tiempoFinMaquina $\leftarrow \mathbf{0}_{|\mathcal{M}|}$
  para cada nivel de operaciones L en S.`ops_by_level` **hacer**
    para cada operación $o \in L$ **hacer**
      $j \leftarrow S$.`op_to_job`$[o]$
      $m \leftarrow \mathbf{P}[c, o]$
      $p \leftarrow S$.`proc_times`$[o, m]$
      $inicio \leftarrow \max\{\mathbf{tiempoFinTrabajo}[j], \mathbf{tiempoFinMaquina}[m]\}$
      $fin \leftarrow inicio + p$
      tiempoFinTrabajo$[j] \leftarrow fin$
      tiempoFinMaquina$[m] \leftarrow fin$
    fin para
  fin para
  $\mathbf{F}[c] \leftarrow \max_{m \in \mathcal{M}} \mathbf{tiempoFinMaquina}[m]$
fin para
devolver F

La selección, el cruce y la mutación se definen como sigue:

1. **Selección por torneo**: para cada individuo, se eligen $k = 3$ competidores al azar y se devuelve el de menor makespan.
2. **Cruce uniforme**: si $\mathbf{p}_1$ y $\mathbf{p}_2$ son dos padres, para cada posición o se genera un bit $b_o \sim$ Bernoulli$(1/2)$; el hijo 1 toma $\mathbf{p}_1[o]$ si $b_o = 1$ y $\mathbf{p}_2[o]$ en caso contrario, y el hijo 2 toma el valor complementario.
3. **Mutación balanceada por carga**: para cada individuo seleccionado con probabilidad p_m, se calcula la carga acumulada de cada máquina; se identifica la máquina con mayor carga y la menos cargada; se elige aleatoriamente una operación asignada a la máquina sobrecargada y se reasigna a la máquina menos cargada.

Con esta formulación, la población actualizada cumple la regla recursiva

$$P_{g+1} = \text{Mutacion}\left(\text{Cruce}\left(\text{Seleccion}(P_g)\right)\right),$$

y la historia de convergencia queda definida por

$$H_g = \left(\min_c C_{\max}(x_c^{(0)}), \ldots, \min_c C_{\max}(x_c^{(g)})\right).$$

Este pseudocódigo refleja de manera fiel la lógica implementada en el motor del algoritmo genético del proyecto, donde la representación de la solución es un vector de asignaciones de máquina por operación y la evaluación se realiza mediante una simulación de cronograma con precedencias y tiempos de máquina.

Esta estructura es típica de los algoritmos evolutivos aplicados a problemas NP-hard, donde la complejidad combinatoria impide una enumeración exhaustiva de soluciones.

8 Conclusiones

El problema de planificación de tareas es un caso clásico de optimización combinatoria con alta complejidad y fuerte dependencia de las restricciones del dominio. Los algoritmos genéticos son una herramienta adecuada para resolverlo porque combinan exploración global con adaptación local, sin requerir una formulación diferencial o convexa del problema.

El sistema desarrollado encapsula este enfoque en una arquitectura moderna con frontend, backend, contenedores y procesamiento distribuido. La principal fortaleza del diseño es su capacidad para representar escenarios reales y explorar soluciones factibles mediante una búsqueda evolutiva guiada por una función de fitness basada en el makespan.

El documento presentado sirve como base teórica y técnica para comprender no solo la idea central del proyecto, sino también los fenómenos matemáticos y computacionales que hacen posible la optimización de problemas de planificación con algoritmos bioinspirados.

Anexos: Ejemplos de prueba y diagramas de Gantt

La carpeta `example_scenarios` contiene un conjunto de instancias de prueba diseñadas para evaluar la calidad, la robustez y la escalabilidad del algoritmo genético. Cada ejemplo representa un problema de programación de tareas con un número distinto de trabajos, operaciones y máquinas, y sirve como referencia para validar el comportamiento del sistema en distintos niveles de complejidad.

Definición de los ejemplos incluidos

1. **Escenario 1: caso pequeño de validación.** Esta instancia está compuesta por 6 trabajos, 5 operaciones por trabajo y 4 máquinas. Tiene un tamaño reducido, de manera que resulta útil para depurar el flujo de decodificación, la evaluación del makespan y la validación de restricciones de precedencia.
2. **Escenario 2: caso de mediana complejidad.** Se trata de una instancia con 10 trabajos, 8 operaciones y 10 máquinas. Su nivel de complejidad es mayor, por lo que permite observar el comportamiento del algoritmo ante cargas de trabajo más exigentes y evaluar la estabilidad de la convergencia.
3. **Escenario 3: caso de prueba de escalabilidad.** Este escenario incluye 12 trabajos, 15 operaciones y 5 máquinas, y representa una prueba de mayor tamaño para evaluar la capacidad del sistema para manejar soluciones factibles con una estructura de conflicto más elevada.

Cada uno de estos ejemplos se almacena en un archivo JSON con el formato de trabajo, flujo de operaciones y tiempos de procesamiento por máquina. En términos estructurales, un escenario se compone de:

- una lista de trabajos $J = \{J_1, \ldots, J_n\}$;
- para cada trabajo, una secuencia de operaciones O_{jk};
- tiempos de procesamiento asociados a cada máquina M_r;
- una representación compatible con la carga del sistema para la generación de soluciones y la evaluación del makespan.

Estos ejemplos constituyen la base experimental del proyecto y permiten comparar el rendimiento del algoritmo ante instancias con distinta dimensión, conflicto y riqueza de restricciones. En una etapa posterior, cada escenario puede complementarse con diagramas de Gantt que visualicen la secuencia de ejecución y los tiempos de finalización obtenidos por la solución propuesta.

Escenario 1

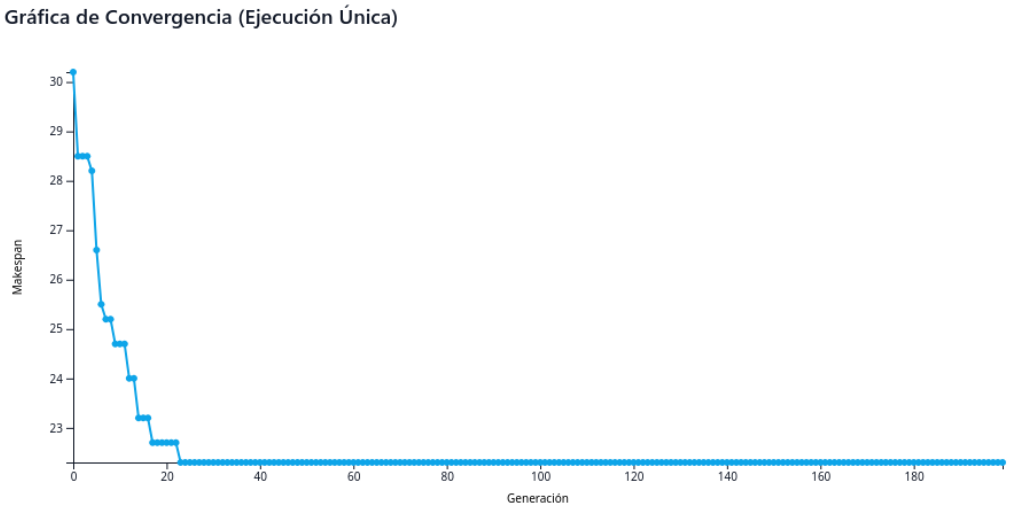


Figura 1: Gráfica de convergencia para el escenario 1.

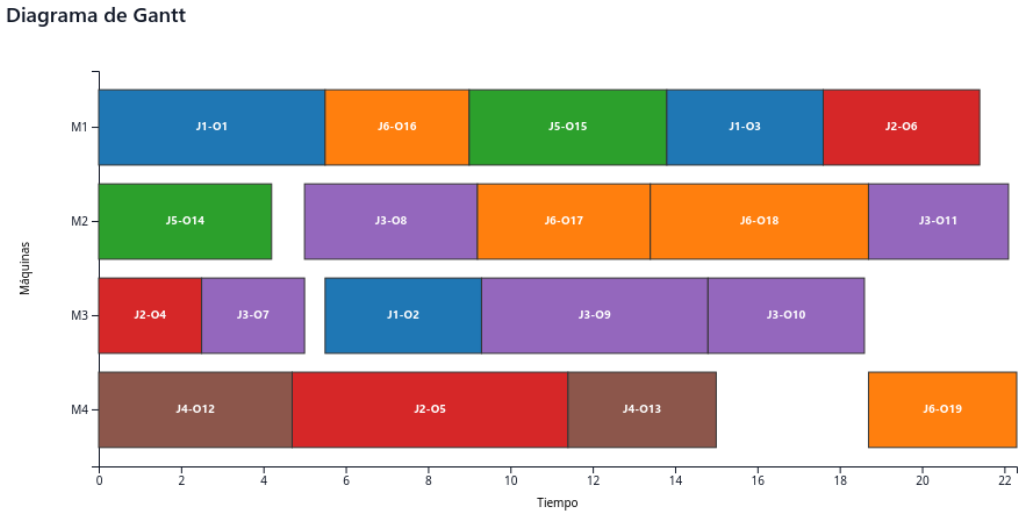


Figura 2: Diagrama de Gantt para el escenario 1.

Escenario 2

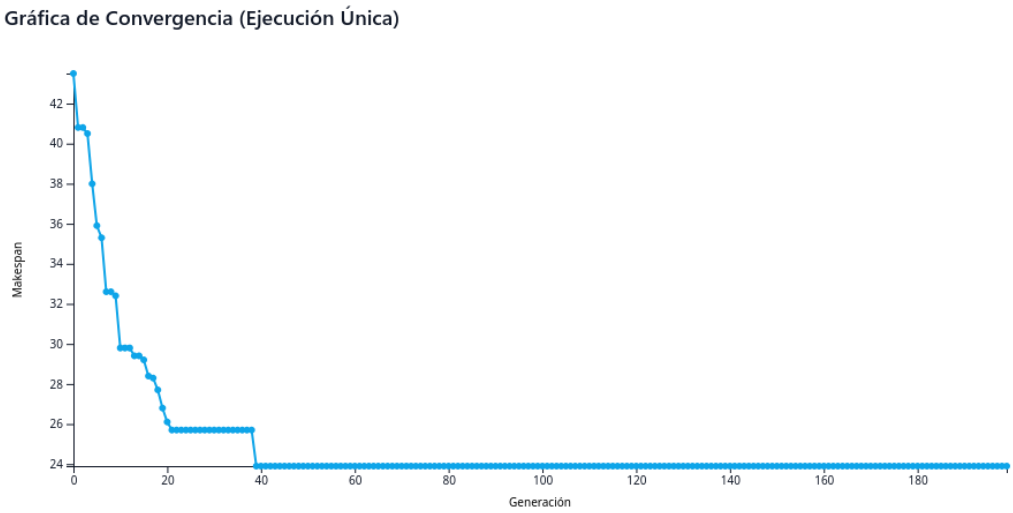


Figura 3: Gráfica de convergencia para el escenario 2.

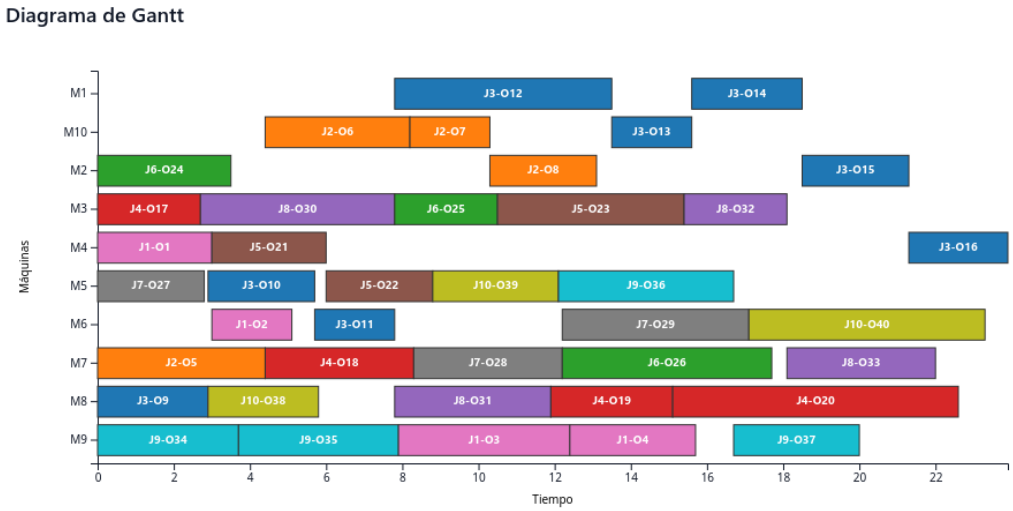


Figura 4: Diagrama de Gantt para el escenario 2.

Escenario 3

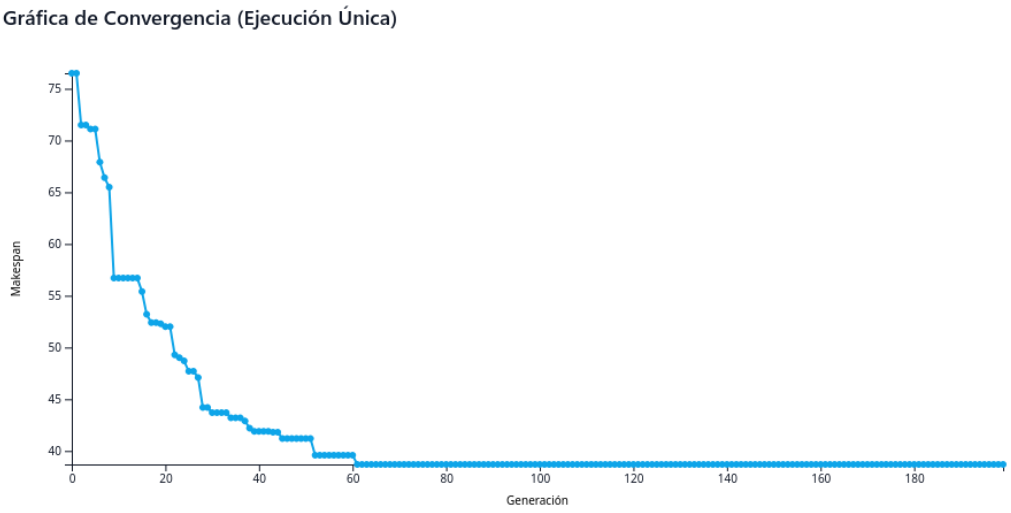


Figura 5: Gráfica de convergencia para el escenario 3.

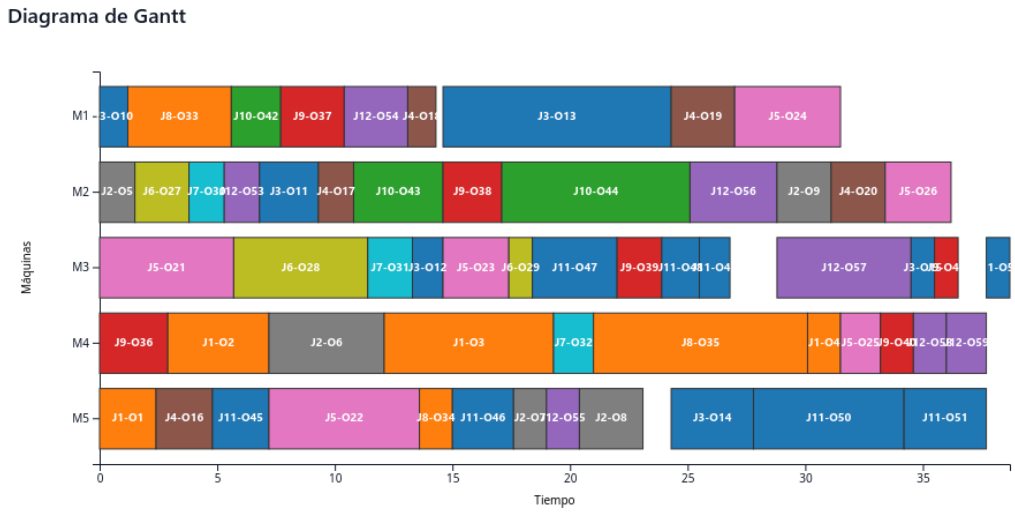


Figura 6: Diagrama de Gantt para el escenario 3.

Referencias

[1] C. A. Coello Coello, "Introducción a la computación evolutiva (notas de curso)," Curso impartido, Av. Instituto Politécnico Nacional No. 2508, Col. San Pedro Zacatenco, México, D.F. 07300, 2026, email: carlosccoellocello@cinvestav.mx.